

EXPERIMENT 16 – Feed Forward Neural Network

Aim

To implement a **Feed Forward Neural Network (FNN)** using Python.

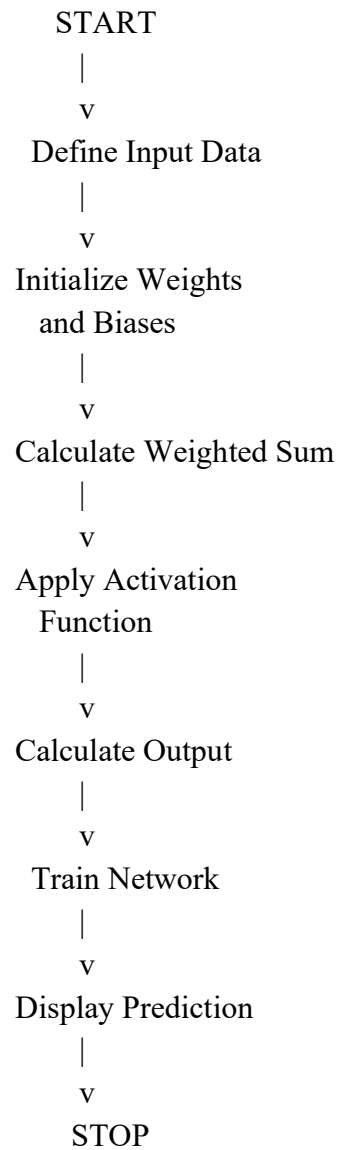
Objective

- To understand the basic structure of a neural network.
- To implement forward propagation.
- To classify simple input data using a neural network.

Algorithm

1. Import the required libraries.
2. Define the input and output training data.
3. Initialize weights and biases randomly.
4. Calculate the weighted sum.
5. Apply the sigmoid activation function.
6. Calculate the output.
7. Repeat the training process for several epochs.
8. Display the predicted output.

Flowchart



Program

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

X = np.array([[0,0], [0,1], [1,0], [1,1]])
Y = np.array([[0], [1], [1], [1]])
```

```

np.random.seed(1)
W = np.random.rand(2,1)
b = np.random.rand(1)

for i in range(10000):
    z = np.dot(X, W) + b
    O = sigmoid(z)

    error = Y - O
    W += np.dot(X.T, error * O * (1-O)) * 0.1
    b += np.sum(error * O * (1-O)) * 0.1

print("Predicted Output:")
print(np.round(O))

```

Output

```

[1]
✓ 0s
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
X = np.array([[0,0], [0,1], [1,0], [1,1]])
Y = np.array([[0], [1], [1], [1]])
np.random.seed(1)
W = np.random.rand(2,1)
b = np.random.rand(1)
for i in range(10000):
    z = np.dot(X, W) + b
    O = sigmoid(z)
    error = Y - O
    W += np.dot(X.T, error * O * (1-O)) * 0.1
    b += np.sum(error * O * (1-O)) * 0.1
print("Predicted Output:")
print(np.round(O))

... Predicted Output:
[[0.]
 [1.]
 [1.]
 [1.]]

```

Result

Thus, the **Feed Forward Neural Network** was successfully implemented using Python.

Conclusion

The neural network successfully learned the given training data and produced the expected classification results.